Installing and Using Redux

We first need to install Redux, and the Redux binding for React:

```
$ yarn add @reduxjs/toolkit react-redux
```

The current recommended way to use Redux is through the toolkit package at `@reduxjs/toolkit`, which provides a more useful API for creating Redux reducers. The core Redux package and the TypeScript definitions will be loaded as a dependency of these packages automatically.

I should note that the new version of Redux triggered TypeScript errors that I was only able to clear by adding the setting `"allowSyntheticDefaultImports": true` to the `compilerOptions` section of `tsconfig.json`—I'm not completely sure why this only triggered on the newest versions, but it has to do with allowing lines like `import React from "react"` with the `as *` part, so I think there's somewhere internally in Redux that does this. (We'll talk more about compiler options in Chapter 13, *Validating Code with Advanced TypeScript*.)

Now we'll create a Redux store in our app.

Creating a Redux Store

To create a store in our app, we'll use the function provided by Redux Toolkit called `configureStore`, which takes a JavaScript object as an argument with various options. The option most important to us is `configureStore({ reducer: venueReducer })`, where the `reducer` option has a reducer function as its value.

The reducer we've been building in our page so far is almost exactly structured to work in Redux as is. The main difference is in initialization. Our existing reducer derives its initial state from the props passed to the `App` component. Redux, on the other hand, requires its reducers to be able to initialize themselves without any input.

Happily, we can adjust our reducer to behave the way Redux expects with just a